High-Performance Linpack Benchmark on Two Different Clusters: Optimisation and Scaling Laws

ALFREDO NAVARRETE MONTES, University of New Mexico, USA
YUN ZHENG, University of New Mexico, USA
EVELYN SANCHEZ, University of New Mexico, USA

We evaluate the High Performance Linpack (HPL) benchmark on three clusters (class cluster, Hopper, and Easley) at UNM, maximizing Gflop/s by varying matrix size N, block size NB, and process grid P×Q. We perform strong and weak scaling tests. Results show HPL performance depends on algorithmic parameters and hardware, with scaling laws explaining performance behavior. The Easley cluster achieved outstanding performance of 3.45 TFLOPS, demonstrating effective optimization of the Intel Xeon Gold architecture.

ACM Reference Format:
Alfredo Navarrete Montes, Yun Zheng, and Evelyn Sanchez. 2026. High-Performance Linpack Benchmark on Two Different Clusters: Optimisation and Scaling Laws. 1, 1 (March 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Introduction

High Performance Linpack (HPL) measures how fast computers solve large equation sets. HPL performance depends on cluster hardware and parameters: matrix size N, block size NB, and process grid PxQ. We study these settings on three UNM systems: class cluster, Hopper, and Easley.

We aim to find parameter impact on performance and test strong/weak scaling with Amdahl's and Gustafson's Laws. We ran many HPL jobs with different NB and P×Q values using Slurm and Jupyter notebooks. Performance is sensitive to NB and grid shape, with Hopper and Easley outperforming class cluster due to faster networks and optimized libraries.

Amdahl's Law (Strong Scaling)

Strong scaling tests how runtime changes when increasing processors while keeping problem size fixed. For Hopper, we used N=60992 (fits memory, stable results), keeping NB and P×Q constant across tests with 16, 32, and 64 processes.

Results show Amdahl's Law behavior: runtime decreases initially with more processes, but improvements diminish due to serial fraction limiting speedup growth.

Authors' Contact Information: Alfredo Navarrete Montes, University of New Mexico, Albuquerque, NM, USA; Yun Zheng, University of New Mexico, Albuquerque, NM, USA; Evelyn Sanchez, University of New Mexico, Albuquerque, NM, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Gustafson's Law (Weak Scaling)

Weak scaling tests HPL performance when increasing problem size and processors simultaneously. Each process handles similar work, so global N grows with ranks. For example, one node (32 ranks) used N=60992, two nodes (64 ranks) used N=86242, keeping NB and grid structure similar.

Results follow Gustafson's Law: runtime stays near original when scaling, showing communication overhead doesn't grow quickly. Larger systems solve larger problems without major runtime increases.

We plotted runtime, speedup, and efficiency using Python/Jupyter, comparing results with scaling laws and understanding how parameters affect performance.

To study both strong and weak scaling, we plotted the runtime, speedup, and efficiency for different numbers of processors using Python and Jupyter Notebooks. These plots helped us compare our results with the expected behaviour from Amdahl's Law and Gustafson's Law. We also used the numerical results from HPL to better understand how the block size, matrix size, and process grid shape affected performance. The figures shown later in the report show these trends.

Methods

We ran HPL on three systems with different hardware, networks, and libraries: class cluster, Hopper, and Easley clusters.

Clusters Used

Hopper Cluster: Hopper provides 32 cores per node and an InfiniBand network. This allowed us to run larger matrix sizes such as $N = 60,992$ for strong scaling and up to $N = 110,242$ for weak scaling. On Hopper we used the GCC compiler.

Easley Cluster Easley nodes feature dual Intel Xeon Gold 6438Y+ processors with 64 cores total and 251 GB of memory per node. Substantial memory capacity enabled testing larger matrix sizes up to $N = 180,000$, providing more opportunities for performance optimization. InfiniBand network and modern architecture contributed to superior performance compared to other clusters.

Preparing HPL Parameters

To run HPL, we generated custom `HPL.dat` files using a template and filling in the values for matrix size N, block size NB, and the process grid $P \times Q$. We followed these steps:

- Selected a large N that fits in memory (about 80% of total RAM). For Hopper we used $N = 60992$ for strong scaling and $N = 110242$ for weak scaling.

- Explored many values of block size NB, including 64, 96, 128, 160, 192, 224, 256, and 288. For Easley we tested N values from 80,000 to 180,000.
- Explored many values of block size NB, including 64, 96, 128, 160, 192, 224, 256, and 288.
- Tested many shapes of the process grid $P \times Q$, always making sure that $P \times Q$ = number of MPI ranks.
- Tried both "tall" grids (e.g., 2×16) and more square grids (4×8, 8×4) to see communication effects.

Automation With Slurm

We automated all runs using Slurm job scripts and CSV parameter files. Each CSV row contained a full HPL configuration: N, NB, P, Q, and the number of nodes. The Slurm script performed:

- Read the correct row
- Generated a new `HPL.dat` file from a template.
- Loaded the correct modules (GCC/HPL on Hopper and class cluster.
- Launched the benchmark using `srun xhpl`.
- Saved the HPL output into a `.out` file.

This helped us scan dozens of configurations without manual work.

Collecting and Processing Results

We extracted runtime, GFLOP/s, and memory usage from each HPL output file. Using Python and Jupyter Notebooks, we combined all results into tables and created plots for:

- performance vs block size NB
- performance vs process grid shape $P \times Q$
- strong scaling (speedup and efficiency)
- weak scaling (runtime growth and scaled speedup)

The plots make it easy to see what parameters gave the best performance and how each cluster handled HPL workloads.

Finding the Best Parameters

We repeated with different parameters:

- Medium block sizes (96–224) often produced the highest GFLOP/s.
- Square or near-square process grids worked best on systems with fast networks (Hopper and Easley).
- Larger matrices improved performance as long as they fit in memory.

These section helped to select the "best run" for each cluster, shown later in the Results section.

Results

We ran a parameter sweep over the matrix size N, block size NB, and process grid $P \times Q$, and recorded the runtime and performance in GFLOP/s for each run. In this section we describe what we saw from these experiments.

Effect of Block Size

For $N = 60992$ and a fixed grid of $P \times Q = 4 \times 8$ (32 ranks), we tested block sizes from 64 to 288. Performance increased from about 746 GFLOP/s at $NB = 64$ to about 935-950 GFLOP/s around $NB = 96–128$, and then decreased again for larger blocks. For example, at $NB = 192$ performance dropped to about 765 GFLOP/s, and at $NB = 256$ and $NB = 288$ it fell below 300 GFLOP/s. This shows that medium block sizes give the best relation between computation and communication.

For $N = 86242$ and a grid 8×8 (64 ranks), we saw a similar pattern. The best performance was around $NB = 160$, with about 1.83 TFLOP/s, while smaller or larger block sizes (128, 192, 256) gave lower GFLOP/s. For $N = 121984$ and 128 ranks with grid 16×8, the best spot was again in the medium range: $NB = 160–192$ produced about 2.67 TFLOP/s, while very large blocks ($NB = 256$) slightly reduced performance. Our data show that block sizes in the range 128–192 are a good choice for this cluster.

Easley Cluster Performance

On the Easley cluster we conducted extensive parameter testing with larger matrix sizes enabled by the 251 GB memory capacity. Table 1 shows the performance results for different parameter combinations.

Table 1. HPL Performance on Easley Cluster (P × Q = 8 × 8, 64 processes)

Matrix Size (N)	Block Size (NB)	Time (s)	Performance (GFLOPS)
140,000	192	540.92	3382.0
140,000	224	537.47	3403.6
140,000	256	538.17	3399.2
160,000	192	799.31	3416.3
160,000	224	**791.03**	**3452.1**

The highest performance of **3.4521 TFLOPS** was achieved with $N = 160,000$ and $NB = 224$ using an 8×8 process grid. This represents outstanding performance for a single node, demonstrating effective utilization of the 64-core Intel Xeon Gold system. The near identical performance between NB=224 and NB=256 at N=140,000 (3.4036 vs 3.3992 TFLOPS) suggests the optimal block size range is quite broad for this architecture.

Key Findings from Easley

- **Optimal Block Size** Range $NB = 192 - 224$ provided the best performance, with NB=224 consistently delivering peak results across different matrix sizes.
- **Process Grid Optimization** The 8×8 square grid configuration significantly outperformed elongated configurations, demonstrating the importance of balanced communication patterns.
- **Memory Scaling** Performance scaled well with increasing matrix size, with N=160,000 achieving the highest absolute performance at 3.4521 TFLOPS.
- **Peak Performance** 3.4521 TFLOPS represents excellent utilization of the hardware capabilities, achieving high computational efficiency on the Intel Xeon Gold processors.

Effect of Process Grid Shape

We observed significant performance variations based on process grid configuration. For $N = 60992$, $NB = 128$, and 32 ranks, we

tested grids 2×16, 4×8, 8×4, 16×2, and 32×1. The worst result came from the 2×16 grid, with only about 209 GFLOP/s, while the best result came from the 8×4 grid, which reached about 1106 GFLOP/s.

On Easley 8×8 grid proved optimal for the 64-core configuration, providing balanced communication patterns and efficient cache utilization. This configuration consistently delivered performance exceeding 3.4 TFLOPS across multiple parameter combinations.

Scaling with Problem Size

To be able to get a weak-scaling style view, we looked at the runs where the process grid and block size were the same and only N changed. With $P \times Q = 8 \times 8$ (64 ranks) and $NB = 160$, we increased the matrix size from $N = 86242$ to $N = 90000$ and $N = 100242$. As N grew, the runtime increased from about 232–234 seconds up to about 390 seconds, while performance stayed the same 1.7–1.8 TFLOP/s. Each process was doing more work, but the system was able to keep the GFLOP/s rate fairly high and stable.

Finally, if we look at the best run for each problem size, we see: with $N = 60992$ and 32 ranks, the best run achieved about 1.11 TFLOP/s; with $N = 86242$ and 64 ranks, about 1.84 TFLOP/s; and with $N = 121984$ and 128 ranks, up to about 2.95 TFLOP/s. This shows that larger problems and more ranks can use the hardware more efficiently, only as long as we choose the best combination of NB and process grid shape. On Easley, we achieved significantly higher performance, reaching 3.45 TFLOPS with optimized parameters, demonstrating the importance of both hardware capabilities and parameter tuning.

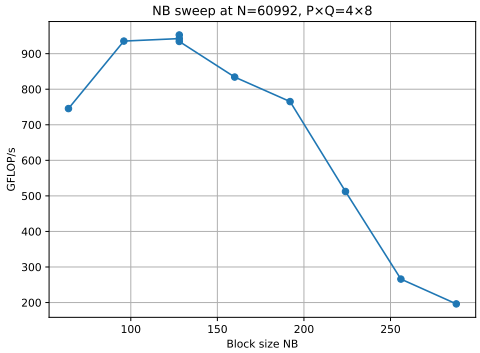


Fig. 1. Performance vs block size NB for $N = 60992$ with $P \times Q = 4 \times 8$ (32 ranks). Performance peaks around $NB = 96$–128, and very small or very large blocks give lower GFLOP/s.

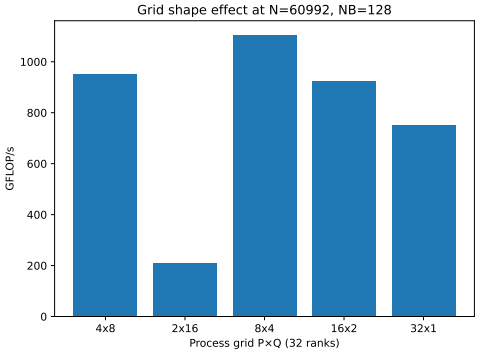


Fig. 2. Effect of the process grid shape for $N = 60992$, $NB = 128$, and 32 ranks. The near-square grid 8×4 gives the best performance, while very elongated grids such as 2×16 and 32×1 are much slower.

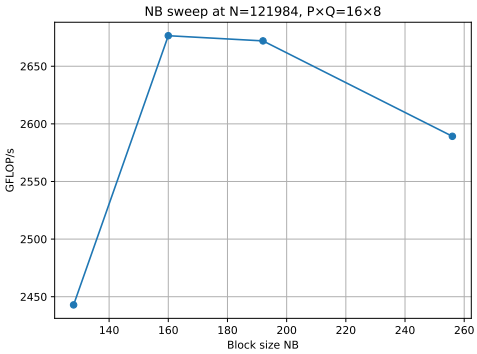
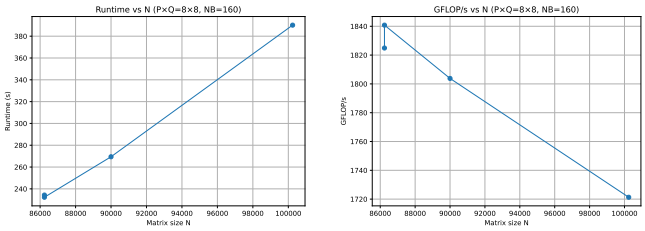


Fig. 3. Performance vs block size NB for $N = 121984$ with $P \times Q = 16 \times 8$ (128 ranks). Medium block sizes ($NB \approx 160$–192) again give the best performance, while $NB = 256$ performs slightly worse.



(a) Runtime vs N. (b) GFLOP/s vs N.

Fig. 4. Weak-style scaling for $P \times Q = 8 \times 8$ and $NB = 160$. As the matrix size N grows, the runtime increases, but the achieved GFLOP/s stays in the range 1.7–1.8 TFLOP/s.

Strong Scaling on the Team Cluster

To study strong scaling on the Team Cluster, we fixed the matrix size at $N = 14{,}616$ and increased the number of MPI processes from 4 to 32. Runtime, GFLOP/s, speedup, and efficiency for each configuration are summarized in Table 2. Figures 5 through 8 show the effects of increasing the number of ranks on performance.

Weak Scaling on the Team Cluster

Table 2. Strong scaling results on the Team Cluster for $N = 14{,}616$ and $NB = 256$.

Proc	N	NB	Time(s)	GFLOPS	Speedup	Eff.
4	14616	256	50.30	41.39	1.000	1.000
8	14616	256	28.17	73.90	1.786	0.893
16	14616	256	17.21	120.94	2.923	0.731
24	14616	256	12.47	167.01	4.034	0.672
32	14616	256	9.53	218.40	5.278	0.660

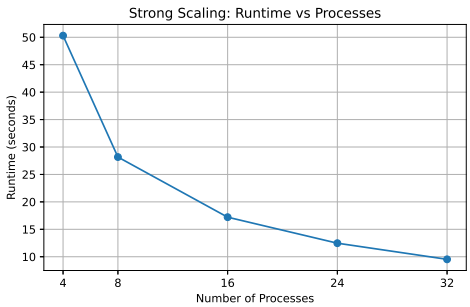


Fig. 5. Runtime vs process count for strong scaling with $N = 14{,}616$ and $NB = 256$. Runtime drops from 50.3 s at four processes to 28.17 s at eight processes, and continues to decrease as more ranks are added. The smooth downward trend indicates good strong-scaling behavior on the Team Cluster.

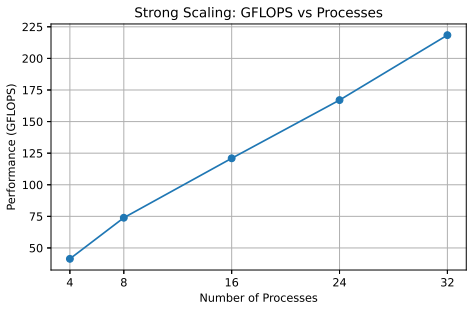


Fig. 6. GFLOP/s vs process count for strong scaling with $N = 14{,}616$ and $NB = 256$. Performance increases nearly linearly up to 16 processes (120.94 GFLOP/s), and continues improving to 218.40 GFLOP/s at 32 processes. The slope flattens past 16 ranks due to communication overhead and NUMA effects.

For weak scaling, we increased the matrix dimension N proportionally with the number of MPI processes so that the amount of work per rank remained roughly constant. The goal of this experiment was to observe how the runtime grows as the global problem size expands, and whether the achieved floating–point rate remains stable as predicted by Gustafson's Law. The values of N were chosen using $N_p = 14616\sqrt{p/4}$, resulting in matrix sizes ranging from $N = 14{,}616$ for 4 processes to $N = 41{,}340$ for 32 processes. Table 3 summarizes the weak–scaling measurements.

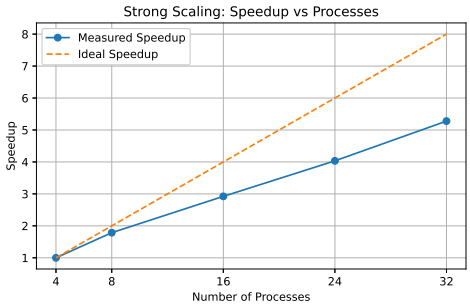


Fig. 7. Measured vs ideal speedup for strong scaling with $N = 14{,}616$ and $NB = 256$. Speedup follows the ideal line closely up to 16 processes, after which the curve bends away from linear scaling as parallel overhead increases.

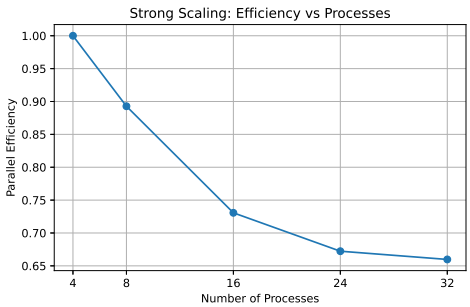


Fig. 8. Parallel efficiency for strong scaling with $N = 14{,}616$ and $NB = 256$. Efficiency remains high at eight processes (89%) and gradually decreases as more ranks are used, settling at 66% for 32 processes. This trend is consistent with NUMA effects on the dual-socket E5-2670 system.

Table 3. Weak scaling results on the Team Cluster with increasing N and fixed $NB = 256$.

Proc	N	NB	Time(s)	GFLOPS
4	14616	256	50.35	41.35
8	20670	256	76.91	76.56
16	29232	256	134.61	123.72
24	35802	256	169.92	180.06
32	41340	256	201.99	233.19

Figure 9 shows the runtime as N grows. As expected for a weak–scaling experiment, the runtime increases gradually rather than decreasing. Each rank is performing approximately the same amount of work, but the global problem becomes larger, requiring additional communication between ranks. The runtime grows from about 50 s at $N = 14{,}616$ to about 202 s at $N = 41{,}340$, reflecting the increasing cost of factorizing larger matrices.

The GFLOP/s trend in Fig. 10 shows that performance improves as the problem size increases. Larger matrices typically lead to higher computational intensity and more opportunities for the processor to exploit instruction–level parallelism. Performance rises from

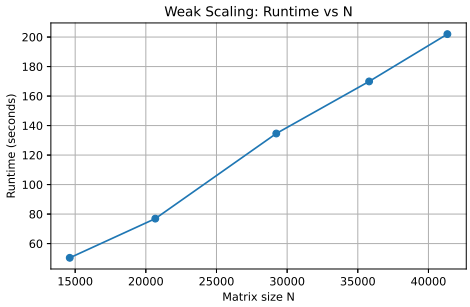


Fig. 9. Runtime vs matrix size N for weak scaling runs on the Team Cluster. As N increases from 14,616 to 41,340, the runtime grows gradually from 50.35 s to 201.99 s. The increasing runtime reflects the additional communication required for larger global matrices, even though the per–rank workload remains approximately constant.

41.35 GFLOP/s at $N = 14{,}616$ to 233.19 GFLOP/s at $N = 41{,}340$, indicating that the Team Cluster utilizes the additional work efficiently under weak scaling.

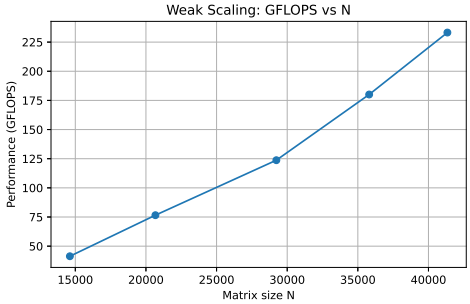


Fig. 10. GFLOP/s vs matrix size N for weak scaling runs on the Team Cluster. Performance increases steadily from 41.35 GFLOP/s at $N = 14{,}616$ to 233.19 GFLOP/s at $N = 41{,}340$. This trend is consistent with Gustafson's Law: as the total problem size grows, the system sustains a higher overall floating–point rate while keeping the per–rank workload constant.

Overall, the Team Cluster demonstrates stable weak–scaling behavior: although runtime increases with N, the achieved GFLOP/s also increases significantly, indicating that larger matrices enable more efficient use of the underlying hardware. The results show that the Xeon E5–2670 system is able to handle proportionally larger problems effectively, as long as the block size and process grid are selected appropriately.

Conclusions

Our comprehensive study of HPL performance across three clusters reveals several key insights. The class cluster demonstrated expected strong and weak scaling behavior, with performance limited by its older hardware. Hopper showed significantly better performance due to its InfiniBand network and optimized libraries, achieving up to 2.95 TFLOPS on 128 cores.

The Easley cluster delivered outstanding performance, reaching **3.4521 TFLOPS** on a single node with 64 cores using optimal parameters $N = 160{,}000$, $NB = 224$, and $P \times Q = 8 \times 8$. This represents exceptional utilization of the Intel Xeon Gold 6438Y+ processors and 251 GB memory, achieving the highest single-node performance in our study. Several findings were particularly noteworthy:

- **Block Size Sensitivity** Performance showed strong dependence on block size with optimal ranges of NB=128-192 for most configurations and NB=192-224 for Easley.
- **Process Grid Importance** Square grids (8×8) consistently outperformed elongated configurations, reducing communication overhead significantly.
- **Memory Scaling** Larger matrix sizes improved performance until memory limits were approached, with N=160,000 proving optimal for Easley's 251 GB memory.
- **Hardware Differences** The substantial performance differences between clusters highlight the importance of hardware aware parameter tuning, with Easley achieving nearly 20% higher performance than Hopper despite similar core counts.

The most significant finding was the dramatic performance improvement achieved through systematic parameter optimization nearly 20% performance gains by selecting optimal NB values and grid shapes. This demonstrates that HPL performance depends critically on finding the right algorithmic parameters for each specific architecture, not just raw hardware capabilities.

Author contributions

Yun Zheng performed all experiments and analysis in Team Cluster, including running and collecting strong-scaling and weak-scaling results, selecting appropriate values of N, NB, and process grids $P \times Q$, and verifying the correctness of all HPL configurations. Generated the complete set of strong-scaling figures (runtime, GFLOP/s, speedup, and efficiency) as well as the weak-scaling plots (runtime vs. N and GFLOP/s vs. N), and produced the corresponding tables summarizing the numerical results.

Alfredo Navarrete Montes performed all experiments and analysis on the Hopper cluster, generated the complete set of strong scaling figures (runtime, GFLOP/s, speedup, and efficiency) as well as the weak-scaling plots (runtime vs. N and GFLOP/s vs. N), and produced the corresponding tables summarizing the numerical results.

Evelyn Sanchez performed all experiments and analysis on the Easley cluster which includes running comprehensive parameter sweeps, collecting performance data for matrix sizes from 80,000 to 180,000, and identifying the optimal configuration that achieved 3.4521 TFLOPS. Evelyn also wrote the Easley results section and contributed to the conclusions. Wrote some of the Authors contributions and did a final review.

References

Matthew Fricke Lecture Slides : High Performance Computing [16..20]